While this helps to search on words (individual or combined), it can only go so far in answering queries such as those with synonyms, plurals, and other elements mentioned earlier. The normalization process takes the analysis from here to the next stage.

7.2.2 Normalization

During *normalization*, tokens (words) are massaged, transformed, modified, and enriched using stemming, synonyms, and stop words. These features are added to the analysis process to ensure that the data is stored appropriately for searching purposes.

One such feature is *stemming*: an operation during which words are reduced (stemmed) to their root words. For example, *author* is a root word for *authors*, *authoring*, and *authored*. In addition to stemming, normalization also deals with finding appropriate synonyms before adding them to the inverted index. For example, *author* has synonyms such as *wordsmith*, *novelist*, and *writer*. And finally, each document will contain words such as *a*, *an*, *and*, *is*, *but*, and *the* that are called *stop words* because they do not have a place in finding relevant documents.

Both tokenization and normalization are carried out by the analyzer module. An analyzer does this by employing filters and a tokenizer. Let's dissect the analyzer module and see what it is made of.

7.2.3 Anatomy of an analyzer

Tokenization and normalization are carried out by three software components—character filters, tokenizers, and token filters—which essentially work together as an analyzer module. As figure 7.1 shows, an analyzer module consists of a set of filters and a tokenizer. Filters work on raw text (character filters) and tokenized text (token filters). The tokenizer's job is to split the sentence into individual words (tokens).

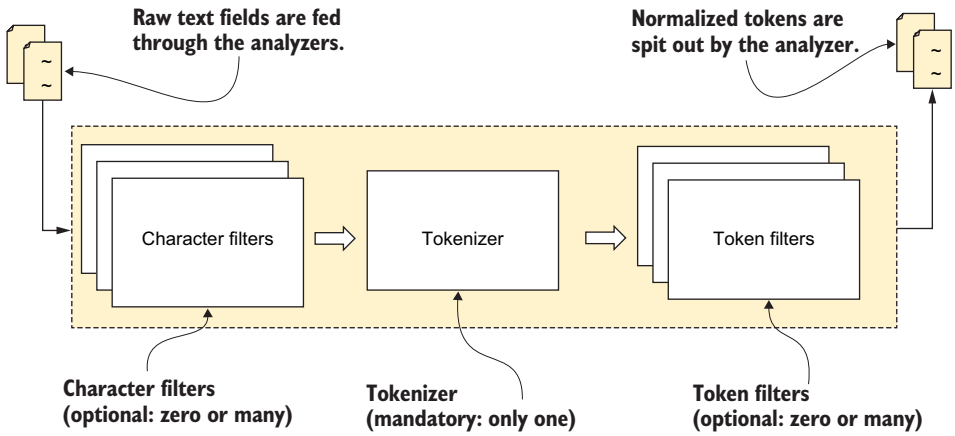


Figure 7.1 Anatomy of an analyzer module